Assignment 1: Self-Supervised Learning with SimSiam on STL-10

B125808026 TAI, WEI-HSUAN

Abstract

In this homework, I re-implement SimSiam [2] with a ResNet-18 backbone and pretrain it on the 100k unlabeled STL-10 images. With only 1,000 labeled images, a linear probe on the frozen SimSiam encoder reaches $80.98 \pm 0.35\%$ test accuracy (200 epochs, 3 pretraining seeds), compared with $47.74 \pm 1.17\%$ for the same network trained from scratch. Removing the stop-gradient or the predictor reproduces the complete collapse reported in the paper: the linear probe drops to exactly chance level (10.00%). As a new ablation, we remove one family of augmentations at a time. Without cropping, SimSiam does not collapse to a single point. Instead it suffers dimensional collapse: the effective rank of the backbone features falls from 318 to 6.7, and the representation gets worse with longer training ($38.2\% \rightarrow 24.8\%$, $p = 0.002$). This shows that stop-gradient and the predictor prevent complete collapse but not dimensional collapse.

1. Setup

Environment. All experiments ran on one machine with a single NVIDIA GeForce RTX 5070 Ti (16 GB), driver 570.211.01 and CUDA 12.8. The software is Python 3.10.21 (conda env mai1), PyTorch 2.8.0+cu128 and torchvision 0.23.0 (Fig. 1). One SimSiam pretraining epoch takes about 45 s, so 100 epochs take about 75 min. A linear probe or a from-scratch run takes 1–2 min.

```
(mai1) josh@josh-ubuntu:HW1$ python -c "import sys,torch,torchvision; print(sys.version); \
print(torch.__version__, torchvision.__version__, torch.version.cuda, \
torch.cuda.get_device_name(0))"
3.10.21 | packaged by conda-forge | (main, Aug 21 2026, 22:42:31) [GCC 14.4.0]
2.8.0+cu128 0.23.0+cu128 12.8 NVIDIA GeForce RTX 5070 Ti
(mai1) josh@josh-ubuntu:HW1$ nvidia-smi
Sun Oct 11 23:34:03 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 570.211.01             Driver Version: 570.211.01     CUDA Version: 12.8     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 5070 Ti     On  |   00000000:01:00.0 Off |                  N/A |
|  0%   35C    P8             19W /  300W |      16MiB /  16303MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            9452      G   /usr/lib/xorg/Xorg                        4MiB |
+-----------------------------------------------------------------------------------------+
```

Figure 1. Environment screenshot: Python, PyTorch and GPU information (nvidia-smi).

Codebase structure. The code extends the official course starter code.

- model.py: the SimSiam model. It contains the ResNet-18 backbone [4] with its fc layer removed (512-d output), a 3-layer projection MLP ($512 \rightarrow 2048 \rightarrow 2048 \rightarrow 2048$, BN on every layer), a bottleneck prediction MLP ($2048 \rightarrow 512 \rightarrow 2048$) and the symmetric negative-cosine loss. The flags stop_grad and predictor={mlp,none} switch the paper's ablations on and off.
- data.py: STL-10 loaders, the two-view augmentation TwoCropsTransform (with aug={full,no_color,no_crop} for our new ablation), and the classifier and evaluation transforms.
- pretrain.py: SimSiam pretraining. After every epoch it records the loss, the kNN accuracy and the output std.
- linear_probe.py (frozen backbone with a linear head; –bn-head adds a parameter-free BN before the head) and sft.py (ResNet-18 trained from scratch).
- utils.py: warmup+cosine LR scheduler, kNN monitor, seeding, CSV/W&B logging.
- Analysis scripts that we added: plot.py and visualize.py (curves, prediction grids, confusion matrices), analyze_rank.py (effective rank), analyze_convergence.py, stats_tests.py (Welch t-tests), summarize_*.py. The experiment drivers are run_*.sh.

Data and protocol. We use STL-10 [3] at 96×96. Pretraining uses the 100k unlabeled images. Classifiers train on the official fold 0 (1,000 images). Validation uses 500 held-out, label-balanced training images (–val-seed 0), and testing uses all 8,000 test images. Pretraining runs SGD (momentum 0.9, weight decay 5×10^{-4}) with batch size 512 and lr $= 0.03 \times 512/256$, using a 10-

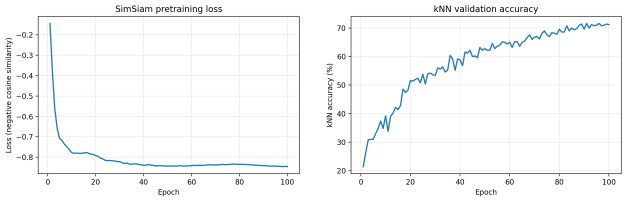


Figure 2. SimSiam pretraining for 100 epochs: training loss (left) and kNN validation accuracy (right).

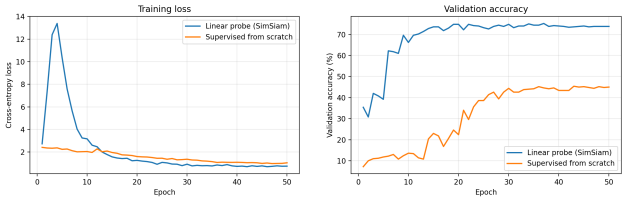


Figure 3. Linear probe on the frozen 100-epoch SimSiam encoder vs. supervised training from scratch on the 1,000 fold-0 images: training loss (left) and validation accuracy (right).

epoch warmup followed by cosine decay. The augmentation follows the paper: RandomResizedCrop (scale 0.2–1), horizontal flip, color jitter $(0.4, 0.4, 0.4, 0.1)$ with p=0.8, and grayscale with p=0.2. The kNN monitor (k=200, τ=0.1) uses fold 0 as the memory bank and evaluates on the 500 validation images. The linear probe runs SGD with lr 30 and no weight decay for 50 epochs. The from-scratch baseline (SFT) uses the pretraining optimizer settings for 50 epochs. Every reported test accuracy uses the checkpoint with the best validation accuracy.

Seeds and statistics. For the 200-epoch comparisons we pretrain with seeds 123/456/789. Each pretrained model is linear-probed with 5 finetuning seeds (12/34/56/78/90). The pretraining seed is the unit of replication: we report the mean ± std over the 3 per-model means and compare settings with two-sided Welch t-tests. When a setting has only one pretraining run, its ± reflects finetuning noise only, and we say so.

2. SimSiam Pretraining (F)

Figure 2 shows a 100-epoch pretraining run. The loss falls quickly from -0.15 to about -0.78 in the first 10 epochs, the warmup period, and ends at -0.848. The kNN accuracy rises steadily to 73.4%. To check for collapse we track the std of the ℓ_2-normalized projector output $z/\|z\|$ (paper Fig. 2). It ends at 0.02195, close to $1/\sqrt{d} = 1/\sqrt{2048} = 0.02210$, the value expected when outputs spread over the unit sphere. The model therefore has not collapsed.

The loss is not a measure of quality. Between epochs 50 and 80 the loss stays flat and even rises slightly ($-0.843 \rightarrow -0.837$). Over the same epochs the kNN accuracy climbs from 60.6% to about 70%. The SimSiam loss only measures agreement between two views, so its value says little about how useful the features are. We return to this point in Sec. 5. Absolute numbers are not comparable to the paper, which uses ImageNet, ResNet-50 and all labels, so we compare only qualitative trends.

Method	Pretrain	Test acc. (%)	n
SFT from scratch	–	47.74±1.17	5 ft. seeds
Linear probe	100 ep	75.80	1 run
Linear probe	200 ep	80.98±0.35	3 pt. seeds
Linear probe	1000 ep	86.74±0.23	1 run†

Table 1. Test accuracy with 1,000 labeled images. †One pretraining run, so ± covers finetuning seeds only.

3. Classification with and without Pretraining (B-)

Both classifiers use the same ResNet-18 and the same 1,000 labeled images (Fig. 3, Tab. 1). The linear probe trains only a 512×10 layer on frozen SimSiam features. It passes 70% validation accuracy around epoch 10 and reaches a much lower training loss. SFT stays near chance for the first 13 epochs and plateaus around 45% validation accuracy. After 200 epochs of pretraining, the linear probe beats SFT by 33.2 points (95% CI 31.8–34.7, Welch t=59.2, p=2.4×10^{-8}). The gap is not caused by too little training of the baselines. Training SFT longer saturates at about 49–50% (100 epochs: 49.62%; 200 epochs: 49.01%, with training loss 0.24, so it overfits). A 200-epoch linear probe adds only +0.8 points over 50 epochs. SimSiam therefore learns from the 100k unlabeled images what 1,000 labels alone cannot teach.

Prediction visualization. In Fig. 4 and the confusion matrix (included in the code submission), the errors are mostly confusions between visually similar animals: cat↔dog, dog↔horse, monkey→cat/dog, deer↔horse. Vehicles (car, ship, truck, airplane) reach 86–93% per-class accuracy. Some mistakes come from context. A sailing ship in front of a building is labeled “cat”, and an airplane against a blue sky is labeled “bird”. Augmentation-invariant features describe global shape and texture well but can be misled by the background.

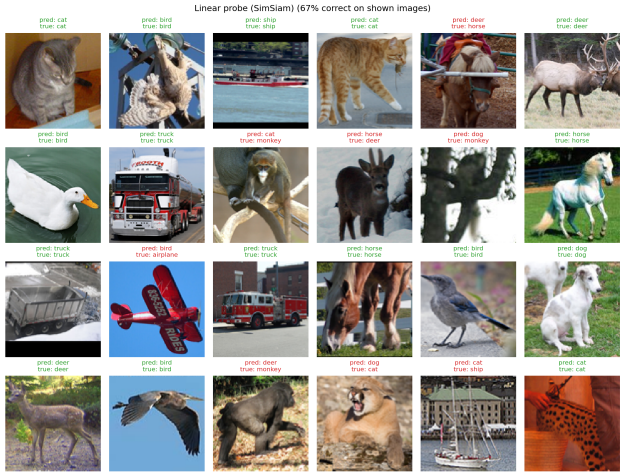


Figure 4. Linear probe predictions on random test images (green: correct, red: wrong). Errors are concentrated among animal classes.

Setting	Loss	Std	kNN	Lin. probe
Baseline	−0.908	0.021	77.6	81.35
w/o stop-grad	−1.000	0	10.0	10.00
w/o predictor	−1.000	0	13.2	10.00

Table 2. Ablations from paper Sec. 4.1–4.2 after 200 epochs (final-epoch values; one run each). After 100 epochs, the two ablations reach 14.89% and 16.16% linear probe accuracy.

4. Paper Ablations: Stop-Gradient and Predictor (B+/A)

We rerun pretraining and the linear probe with (a) the stop-gradient removed (–no-stop-grad, so gradients flow through both branches) and (b) the predictor replaced by the identity (–predictor none). Each variant writes to its own checkpoint and log directory.

Results. Both variants reproduce the paper's collapse (Fig. 5, Tab. 2). Within a few epochs the loss reaches its minimum of −1 and the output std drops toward 0. Every image is mapped to the same vector. After 200 epochs the linear probe scores exactly 10.00% with training loss $\ln 10 = 2.303$, which is chance level for 10 classes. The backbone outputs the same feature for every input. The paper's 0.1% is the same result for ImageNet's 1000 classes. After 100 epochs the probes still reach about 15%, because the backbone has not fully collapsed yet.

Insights. The collapse happens in two stages. In the first 5 epochs the std falls sharply to about 0.003, a nearly collapsed output; between epochs 40 and 100 it decays slowly to 0. The two ablations produce nearly the same curves, which matches the paper's analysis. Without a predictor, the symmetric loss $\frac{1}{2}\mathcal{D}(z_1, \mathrm{sg}(z_2)) + \frac{1}{2}\mathcal{D}(z_2, \mathrm{sg}(z_1))$ has the same gradient as $\mathcal{D}(z_1, z_2)$, so removing the predictor in effect removes the stop-gradient as well. Neither the architecture nor BN prevents the trivial solution. The asymmetry from the stop-gradient and the predictor is required. Because collapse here is deterministic (loss = −1, accuracy exactly at chance), one run per variant is enough.

5. Beyond Expectation: Augmentation Ablation (A+)

Motivation. The paper ablates the architecture (predictor, BN, stop-grad) and the loss (similarity, symmetry) but not the data side. SimSiam has no negative pairs, so the difference between the two augmented views is its only learning signal. Its loss is minimized by anything the two views share. The paper's EM-like hypothesis (Sec. 5) treats SimSiam as approximately solving $\min_\theta \mathbb{E}_{x,\mathcal{T}} \|\mathcal{F}_\theta(\mathcal{T}(x)) - \eta_x\|^2$, where η_x is the expected representation over augmentations $\mathcal{T}$. If the augmentations are too weak, this expectation becomes trivial. SimCLR [1] found crop+color to be the most important augmentation pair for contrastive learning, where negatives provide a separate force against collapse. We test whether this also holds without negatives, and whether stop-gradient and the predictor still protect against collapse when the views become too easy.

Setup. –aug no_color drops color jitter and grayscale. –aug no_crop drops RandomResizedCrop, so both views show the full image and differ only in flip and color. Everything else matches the baseline. We pretrain each setting for 200 epochs with 3 seeds and run 5 probe seeds per model. We measure effective rank (erank) [6] instead of relying only on std. Let σ_k be the singular values of the centered, ℓ_2-normalized features of the 8,000 test images. Then $\mathrm{erank} = \exp(-\sum_k p_k \log p_k)$ with $p_k = \sigma_k / \sum_j \sigma_j$. We focus on the 512-d backbone feature h, which is the input to the linear probe.

Lower loss, worse representation. Both variants reach a lower loss than the baseline, because the views are easier to align, yet their accuracy is far lower (Tab. 3). Removing color costs 30.6 points (p=1.8×10^{-4}). Removing crop costs 56.2 points even under the fairer BN probe (p=1.3×10^{-4}), and no_crop is 25.6 points below no_color (p=3.7×10^{-5}). Cropping is therefore the more important augmentation. This agrees with

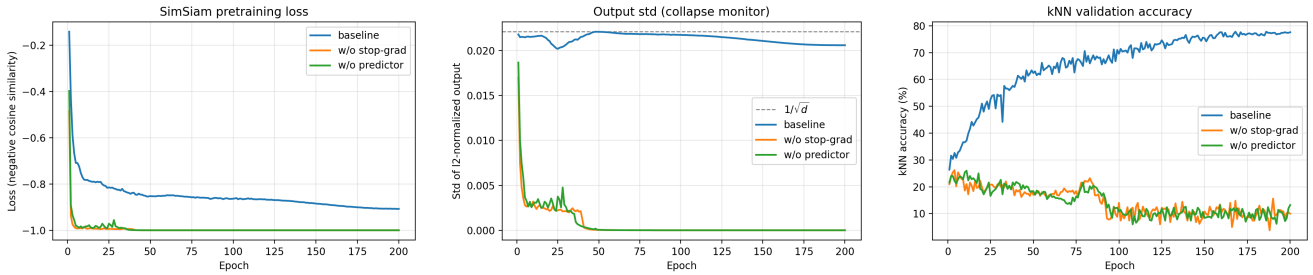


Figure 5. Ablating the stop-gradient and the predictor (200 epochs): training loss, std of the ℓ_2-normalized output (dashed: $1/\sqrt{d}$) and kNN accuracy. Both ablations collapse.

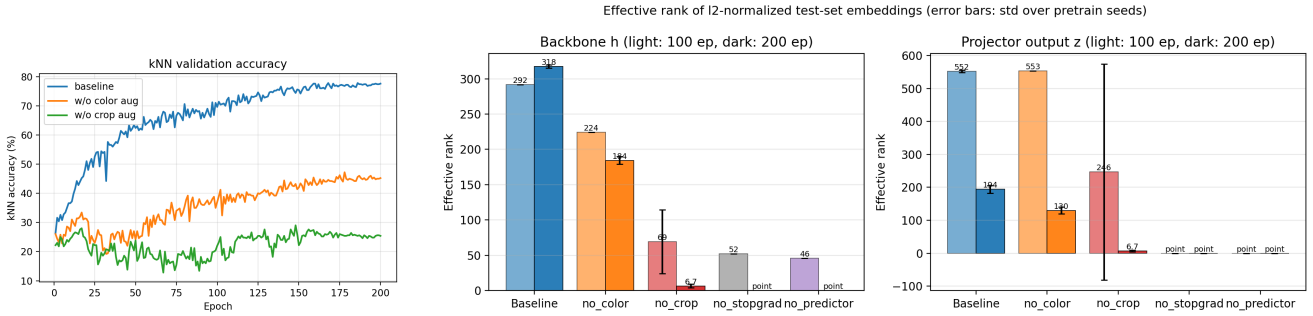


Figure 6. Left: kNN accuracy during augmentation-ablation pretraining (one representative 200-epoch run each). Right: effective rank of test-set embeddings for the backbone h and projector output z after 100 epochs (light) and 200 epochs (dark); error bars show std over pretraining seeds. "point" means all images map to one point.

Setting	Loss	kNN	Lin. probe	BN probe
Baseline	−0.909	78.1	80.98±0.35	82.36†
w/o color	−0.958	44.9	50.36±1.17	51.18†
w/o crop	−0.995	19.3	17.50±2.19	24.77±1.54

Table 3. Augmentation ablation, 200 epochs, mean (±std) over 3 pretraining seeds. Loss and kNN are final-epoch values averaged over seeds. BN probe: parameter-free BN before the linear layer, lr chosen on validation. †Seed 123 only.

SimCLR, but here the drop is larger. Without negatives, a too-easy view pair leaves no other signal to learn from. no_crop is also the least stable setting: its std across pretraining seeds is about 6× the baseline's (F-test $p \approx 0.025$, n=3).

The standard probe fails on no_crop. With lr 30, the linear probe on no_crop features never gets below training loss $\ln 10$, whether trained for 50, 100 or 200 epochs, and its best validation epoch is always among the last few. The no_crop features occupy a tiny subspace with very small variance (see below), so the fixed lr is poorly scaled for them. Adding a parameter-free BN to standardize the features lets the probe train (validation selects lr 1.0, versus 0.03 for the baseline). It raises no_crop to 24.77% (+7.27, paired t=19.2, p=0.003). We therefore report the BN probe for no_crop. The conclusion still holds: no_crop features are poor, about 58 points below the baseline.

no_crop gets worse with more training. no_color improves from 100 to 200 epochs (43.9 → 50.4). no_crop moves the other way. With 3 seeded 100-epoch runs and the BN probe, its accuracy drops from 38.17±2.39 to 24.77±1.54 (Welch t=8.18, df 3.4, p=0.0023; −13.4 points, 95% CI [−18.3, −8.5]). The kNN curve in Fig. 6 (left) never rises steadily, unlike the baseline's.

Dimensional collapse, not complete collapse. The erank explains these results (Fig. 6, right). As a sanity check, the 200-epoch no_stop-grad and no_predictor models are "points": their feature variance relative to the mean squared norm is 10^{-15} or smaller. no_crop is different. Its projector output z still spreads out (relative variance 0.93, std not zero), but it occupies only about 4–8 of 2048 dimensions. The backbone erank is just 6.7±2.8, versus 317.5±2.4 for the baseline. This is dimensional collapse [5]. It also gets worse with train-

ing: for every seed, the backbone erank drops from 100 to 200 epochs (53.3 → 9.2, 120.3 → 7.4, 34.2 → 3.6), at the same time as the accuracy drops. It also explains why the standard probe failed: a few informative directions with tiny scale require a standardized head. Takeaway: stop-gradient and the predictor prevent complete collapse, but they do not prevent dimensional collapse. In terms of the EM view, a crop-free pair differs only in color and flip. Averaging over such augmentations gives an η_x that a few color-invariant directions can match, and the objective keeps shrinking the representation onto those directions.

Limitations and counterexamples. (i) Erank is not a quality measure across settings. For no_color, erank decreases from 100 to 200 epochs (224 → 184) while accuracy increases (43.9 → 50.4). Our evidence for no_crop is strong only because rank and accuracy fall together, and by large amounts. (ii) A falling std is not a sign of collapse either. The 1000-epoch baseline's std halves to 0.0100, yet its backbone erank (392) and accuracy (86.7%) are the highest of all our models. (iii) We use n=3 pretraining seeds, so the Welch df is only 2–4 and the CIs are wide. Even so, every effect above is many standard deviations in size.

6. Supplementary: Training Length

Figure 7 shows that longer schedules keep helping, with diminishing returns: 75.8% (100 epochs), 81.0% (200) and 86.7% (1000). This matches the trend in the paper's Tab. 4. Within the 1000-epoch run, the loss and kNN accuracy plateau from epoch 300 to 500 while the LR is still above 0.5× its peak. A saturation rule (less than 1 point gained per 100 epochs) would stop training at epoch 436, yet the linear probe accuracy keeps rising from 80% to 87% as the LR anneals. The plateau is therefore limited by the LR, not by the representation, and the real saturation happens only in the last 100 epochs (+0.07 points from epoch 950 to 1000). Snapshot accuracy also fluctuates along the trajectory: epoch 700 is 2.0 points below epoch 650, a gap far larger than the std across probe seeds (at most 0.6). A single mid-schedule snapshot therefore does not reliably represent "training up to that epoch".

7. Conclusion

SimSiam on STL-10 turns 100k unlabeled images into features that beat training from scratch by 33 points with only 1,000 labels. We confirmed that removing the stop-gradient or the predictor causes complete collapse. Our new augmentation ablation shows a second failure mode: when the views are too similar (no cropping), SimSiam keeps a nonzero std and a lower loss, yet its features collapse onto a few dimensions and degrade with more training. Neither the loss value nor the output std detects this; the effective rank and a properly scaled linear probe do.

References

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In International Conference on Machine Learning (ICML), pages 1597–1607, 2020. 3

[2] Xinlei Chen and Kaiming He. Exploring simple Siamese representation learning. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 15750–15758, 2021. 1

[3] Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In International Conference on Artificial Intelligence and Statistics (AISTATS), pages 215–223, 2011. 1

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 770–778, 2016. 1

[5] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. In International Conference on Learning Representations (ICLR), 2022. 4

[6] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In European Signal Processing Conference (EUSIPCO), pages 606–610, 2007. 3

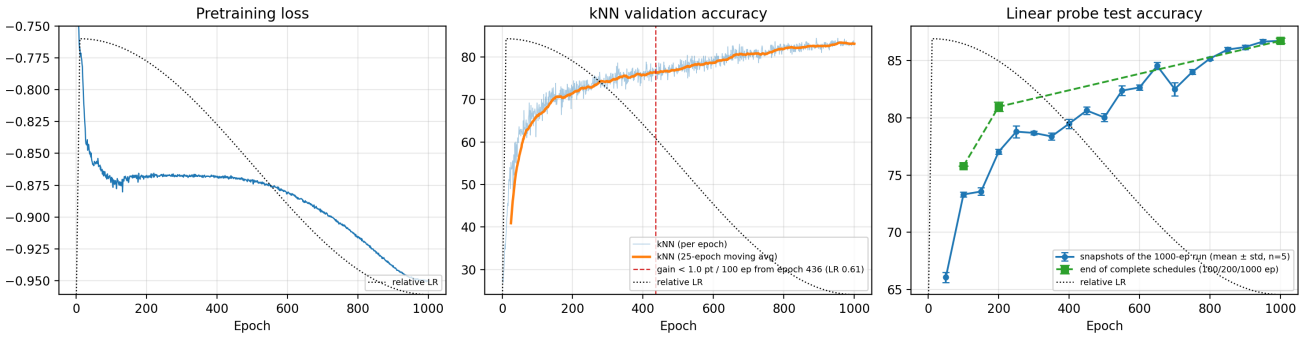


Figure 7. A 1000-epoch baseline run (dotted: relative LR). Right: linear probes on snapshots taken every 50 epochs (mean ± std over 5 probe seeds; green: final accuracy of complete 100/200/1000-epoch schedules).